

Implementación y análisis de complejidad de listas, pilas y colas en Java

Hayran Andrés López González

Objetivo del taller

Implementar las estructuras de datos List (listas enlazadas), Stack (pila) y Queue (cola) en Java, abarcando tanto arreglos dinámicos como listas enlazadas, realizando un análisis de complejidad de los métodos asociados.

Explicación de la implementación

Este trabajo ha sido implementado en el siguiente repo de GitHub:

https://github.com/Andresit1524-Uni/lab1_edd

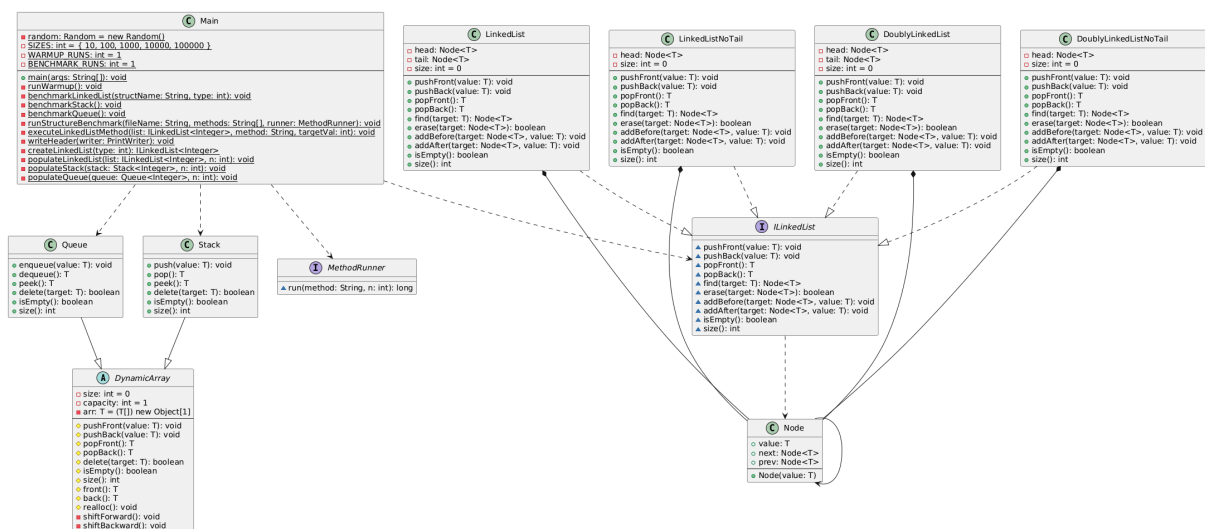
La implementación se rige por la siguiente estructura:

- Una interfaz `ILinkedList` con los métodos comunes para todas las listas enlazadas (insertar, eliminar, buscar, etc.).
- Una clase abstracta `DynamicArray` con los métodos de un arreglo dinámico ya implementados. Es abstracta porque no la usaremos directamente.

De aquí parten las clases que si se usan:

- Las 4 variantes de listas enlazadas, todas implementando la interfaz `ILinkedList`:
 - `LinkedList`
 - `LinkedListNoTail`
 - `DoublyLinkedList`
 - `DoublyLinkedListNoTail`
- `Stack` y `Queue` para las pilas y colas respectivamente, basados en la clase abstracta `DynamicArray`

El diagrama UML de este proyecto es el siguiente:



Análisis de complejidad

Listas enlazadas

Simple

- Buscar siempre toma $O(n)$, y es el cuello de botella de los siguientes métodos
- Insertar y eliminar al frente es inmediato porque tenemos un puntero a la cabeza de la lista y el siguiente nodo. Complejidad $O(1)$
- Insertar y eliminar al final requiere buscar el último nodo y el penúltimo (al eliminar), por lo que la complejidad es $O(n)$
- Añadir antes de un nodo requiere buscar el anterior ($O(n)$) pero añadir después se puede hacer de inmediato porque tenemos acceso al siguiente nodo ($O(1)$)
- Eliminar un nodo requiere buscar el anterior, entonces su complejidad es $O(n)$
- Verificar que está vacío es tan sencillo como ver si hay cabeza. Complejidad $O(1)$

Ten en cuenta que los métodos `addBefore` y `addAfter` ofrecen un puntero directo al nodo, entonces por ello pueden ser más rápidos.

Simple (con cola)

El nuevo puntero con cola hace que añadir elementos al final sea más sencillo porque tenemos la cola directamente. Pero el resto de operaciones siguen igual en cuanto requieren búsquedas (especialmente para buscar el nodo anterior a uno dado).

Dobles

Las listas enlazadas dobles introducen un puntero anterior que nos permite trabajar hacia atrás también. Gracias a eso, ahora borrar nodos y añadir antes de uno puede hacerse en tiempo constante porque podemos ver hacia atrás y cablear directamente.

Pero como no tienen puntero a cola, aún debemos buscarla, degradando las operaciones al final de la lista.

Dobles (con cola)

Ahora con el puntero a cola, podemos ir directo al final de la lista y operar sobre ella, haciendo que la adición/eliminación de elementos se torne de tiempo constante.

Con estas consideraciones, la tabla de complejidades queda como se ve a continuación. En azul las operaciones con rendimiento mejorado y en rojo las que quedan peor a comparación.

Operación	Simple	Simple (con cola)	Doble	Doble (con cola)
<code>pushFront</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>pushBack</code>	$O(n)$	$O(1)$	$O(n)$	$O(1)$
<code>popFront</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>popBack</code>	$O(n)$	$O(n)$	$O(n)$	$O(1)$
<code>find</code>	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<code>erase</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<code>addBefore</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<code>addAfter</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>isEmpty</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Comparando las tablas obtenemos que la mejor estructura en términos de rendimiento general es la **lista enlazada doble con puntero a cola**, al permitir un avance en dos direcciones y acceso a los dos extremos.

Arreglos dinámicos, pilas y colas

Los arreglos dinámicos son arreglos cuyo tamaño puede crecer de manera indefinida, permitiendo acceder a elementos de manera aleatoria (por índice) y reasignarse automáticamente cuando se llena.

Eliminar o añadir un elemento de una posición diferente a la última requiere trasladar el resto de elementos hacia adelante o hacia atrás, lo cual hace que su complejidad sea de $O(1)$. El resto de operaciones pueden hacerse de manera inmediata y por ende con $O(1)$.

La complejidad de sus operaciones es la siguiente:

Operación	Pila	Cola	Complejidad
pushFront	-	-	$O(n)$
pushBack	push	enqueue	$O(1)$
popFront	-	dequeue	$O(n)$
popBack	pop	-	$O(1)$
delete	delete	delete	$O(n)$
isEmpty	isEmpty	isEmpty	$O(1)$
size	size	size	$O(1)$
front	-	front	$O(1)$
back	peek	-	$O(1)$

Ten en cuenta que se están usando arreglos dinámicos llenados de inicio a fin. Con arreglos circulares se puede lograr más rendimiento, y con arreglos circulares dinámicos (no implementados en este caso) también una capacidad máxima arbitraria.

Demostración de que la inserción es $O(1)$

Como se vió en clase, insertar elementos al final de un arreglo toma un tiempo constante. Tomaré mi versión favorita de la demostración:

Gráficación y resultados empíricos

Para analizar el comportamiento real de estos métodos utilizaremos medidas basadas en `System.nanoTime()`. El esquema de medición es el siguiente:

- Mediremos el rendimiento con 10, 100, 1000, ..., hasta 10^5 operaciones
- Esta medición es por cada método de cada clase implementada
- Haremos la tabla de tiempos y gráficas para los mismos. Una tabla por cada estructura de datos
- Las gráficas serán en escala logarítmica para evidenciar todas las escalas adecuadamente. Una gráfica por cada tabla
- Se usarán **nanosegundos** para las mediciones

Funcionamiento de la Suite:

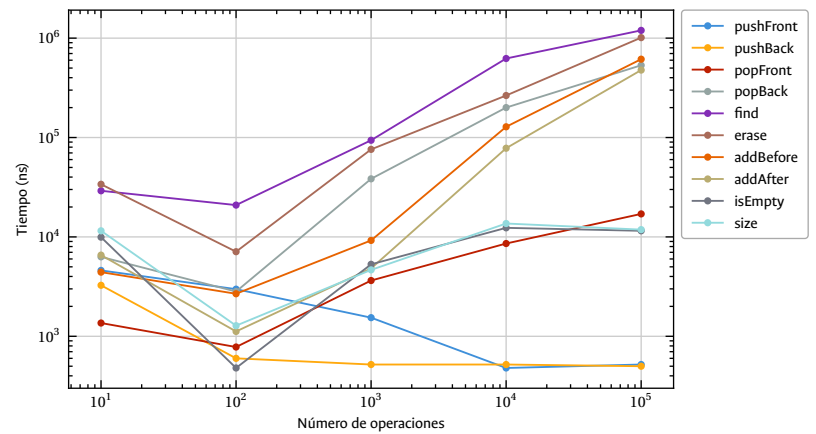
1. **Calentamiento:** Ejecuta pasadas iniciales descartadas para permitir que la JVM realice optimizaciones JIT.
2. **Escala exponencial:** Evalúa cada método con tamaños $n \in \{10, 100, 1000, 10000, 100000\}$.
3. **Población con datos aleatorios:** Se llenan las estructuras previamente usando valores aleatorios para métodos de eliminación o consulta.
4. **Archivos CSV matriciales separados:** Se genera un archivo .csv independiente por cada estructura de datos.

El código para registrar estos tiempos no interfiere con el rendimiento de los algoritmos, y se ubica toda en la clase Main. Las gráficas se crean con **lilaq**, una librería de **Typst**, el mismo sistema con el que se ha compilado este PDF. los datos estarán también registrados en un CSV en data/.

Listas enlazadas

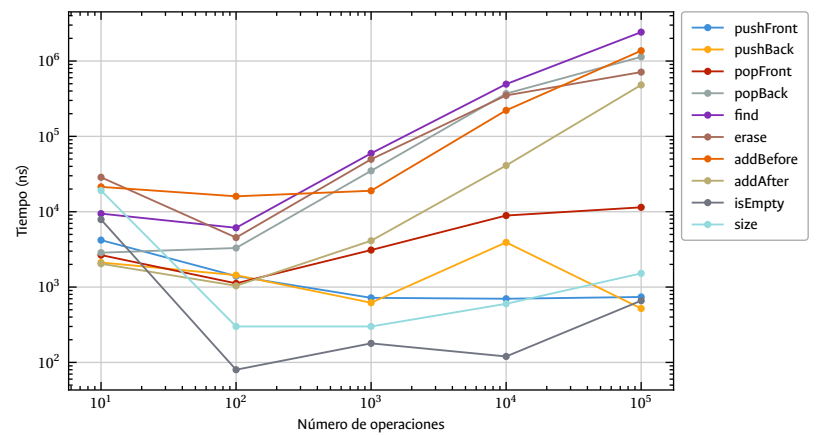
1. Simples

Method	10	100	1000	10000	100000
pushFront	4600	2980	1540	480	520
pushBack	3260	600	520	520	500
popFront	1360	780	3640	8580	17040
popBack	6300	2820	38400	200160	533180
find	29120	20899	93860	623899	1197040
erase	33860	7100	75979	264720	1011820
addBefore	4420	2679	9220	128220	614200
addAfter	6559	1119	4820	78040	477140
isEmpty	9939	480	5300	12339	11520
size	11519	1279	4659	13639	11839



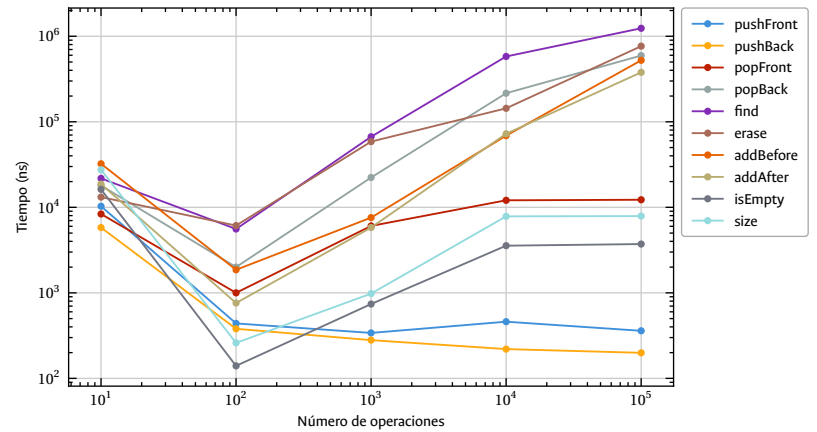
2. Simples (con cola)

Method	10	100	1000	10000	100000
pushFront	4200	1399	720	700	739
pushBack	2120	1440	620	3920	520
popFront	2660	1120	3100	8900	11440
popBack	2860	3300	34900	369179	1136020
find	9440	6119	59520	493760	2418919
erase	28639	4539	49679	349500	713200
addBefore	21380	16019	18979	221500	1370040
addAfter	2040	1040	4120	41080	481860
isEmpty	7879	80	179	120	660
size	19140	300	300	600	1520



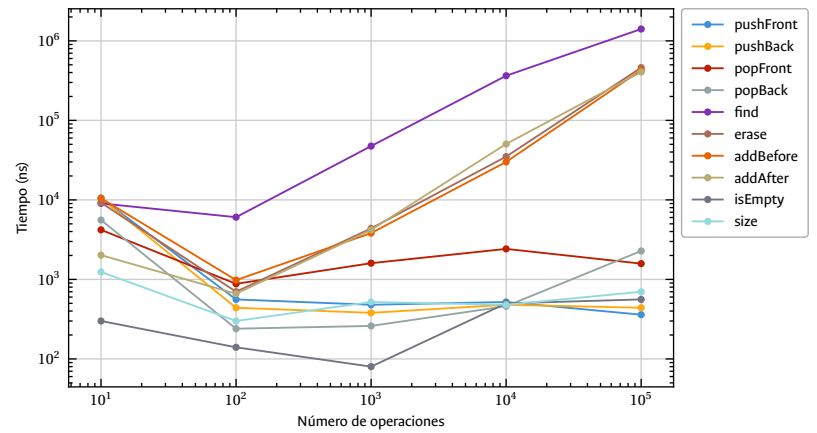
3. Dobles

Method	10	100	1000	10000	100000
pushFront	10300	439	340	460	360
pushBack	5799	380	280	220	199
popFront	8360	1000	6059	12080	12260
popBack	17980	2000	22319	215860	594620
find	21799	5579	66860	580280	1239940
erase	13180	6120	58660	143700	765820
addBefore	32320	1860	7580	68820	523280
addAfter	18680	760	5800	72420	378660
isEmpty	16220	140	740	3560	3720
size	27520	260	980	7840	7900



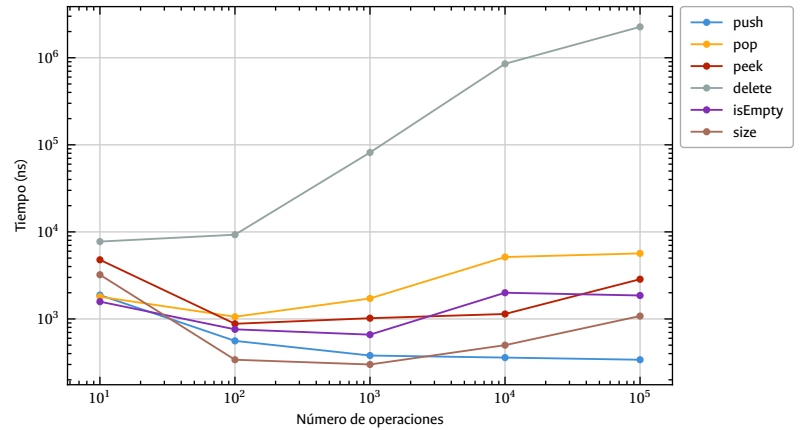
4. Dobles (sin cola)

Method	10	100	1000	10000	100000
pushFront	10540	560	480	520	360
pushBack	10440	440	380	480	440
popFront	4200	880	1600	2420	1580
popBack	5580	240	260	460	2280
find	9040	6060	47520	364320	1409120
erase	9220	700	4380	35020	460440
addBefore	10480	980	3820	30020	429000
addAfter	2020	660	4200	50500	406120
isEmpty	300	140	80	500	560
size	1240	300	520	480	700



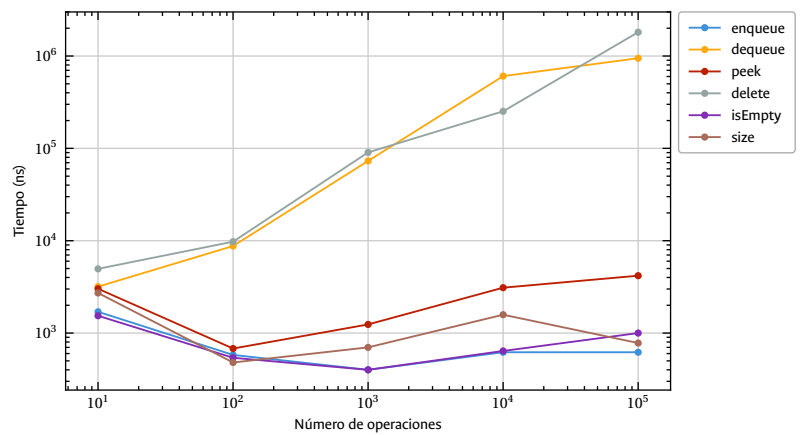
Pilas

Method	10	100	1000	10000	100000
push	1880	560	380	360	340
pop	1800	1060	1720	5140	5660
peek	4780	880	1020	1140	2860
delete	7740	9280	81600	852000	2263400
isEmpty	1580	760	660	2000	1860
size	3220	340	300	500	1080



Colas

Method	10	100	1000	10000	100000
enqueue	1700	580	400	620	620
dequeue	3180	8760	73100	606040	947080
peek	3020	680	1240	3100	4180
delete	4960	9760	90440	251920	1810440
isEmpty	1540	540	400	640	1000
size	2720	480	700	1580	780



Conclusiones

Hemos hecho un análisis teórico y empírico sobre el funcionamiento de las listas enlazadas en todas sus formas, las pilas y las colas y hecho supuestos en base a su funcionamiento y a análisis simples.

La experimentación demostró que la complejidad de las estructuras de datos coincidió con lo esperado y demostrado a lo largo de este informe, gracias a la implementación y medición hechos en Java.

Estos resultados son esenciales en el manejo real de estructuras de datos, las cuales se usan en todos los casos de uso posibles dentro del software real. Su comprensión nos permitirá elegir la mejor estructura para cada contexto.